57. Insert Interval Medium

Acceptance: 44.6% | LeetCode

You are given an array of non-overlapping intervals `intervals` where `intervals[i] = [starti, endi]` represent the start and the end of the `ith` interval and `intervals` is sorted in ascending order by `starti`. You are also given an interval `newInterval = [start, end]` that represents the start and end of another interval.

Insert `newInterval` into `intervals` such that `intervals` is still sorted in ascending order by `starti` and `intervals` still does not have any overlapping intervals (merge overlapping intervals if necessary).

Return `intervals` *after the insertion*.

Note that you don't need to modify `intervals` in-place. You can make a new array and return it.

Example 1:

```
**Input:** intervals = [[1,3],[6,9]], newInterval =
[2,5]
**Output:** [[1,5],[6,9]]
```

Example 2:

```
**Input:** intervals = [[1,2],[3,5],[6,7],[8,10],
[12,16]], newInterval = [4,8]
**Output:** [[1,2],[3,10],[12,16]]
**Explanation:** Because the new interval [4,8]
overlaps with [3,5],[6,7],[8,10].
```

Constraints:

- `0 <= intervals.length <= 104`
- `intervals[i].length == 2`
- `0 <= starti <= endi <= 105`
- `intervals` is sorted by `starti` in **ascending** order.
- `newInterval.length == 2`
- `0 <= start <= end <= 105`

Claude Opus Solution

Insert Interval

1. Intuition

The key insight is that since `intervals` is **already sorted** by start time and **non-overlapping**, we can process intervals in a single pass and categorize each one into exactly **three groups**:

1. **Intervals that come entirely BEFORE** `newInterval` (no overlap, end before newInterval starts)
2. **Intervals that OVERLAP** with `newInterval` (these get merged into one)
3. **Intervals that come entirely AFTER** `newInterval` (no overlap, start after newInterval ends)

Two intervals `[a, b]` and `[c, d]` **overlap** if and only if `a <= d AND c <= b`. When they overlap, the merged interval is `[min(a, c), max(b, d)]`.

The sorted property means once we pass the overlapping region, everything after is guaranteed to be "after" — so we can append the rest directly.

2. Approach

```
1. Initialize an empty result list and pointer i = 0.

2. PHASE 1 — Add all intervals ending before newInterval starts:
   While i < n AND intervals[i].end < newInterval.start:
       append intervals[i] to result
       i++

3. PHASE 2 — Merge all overlapping intervals into newInterval:
   While i < n AND intervals[i].start <= newInterval.end:
       newInterval.start = min(newInterval.start, intervals[i].start)
       newInterval.end   = max(newInterval.end,   intervals[i].end)
       i++
   Append the merged newInterval to result.

4. PHASE 3 — Add all remaining intervals (which start after newInterval ends):
   While i < n:
       append intervals[i] to result
       i++
```

```
5. Return result.
```

---

3. Python Solution

```python
from typing import List


class Solution:
    def insert(self, intervals: List[List[int]], newInterval: List[int]) -> List[List[int]]:
        result = []           # Holds the final merged intervals
        i = 0                 # Pointer to traverse intervals
        n = len(intervals)

        # ---------- PHASE 1: Intervals that come BEFORE newInterval ----------
        # An interval is fully before newInterval if its END is strictly less
        # than newInterval's START (no overlap possible). Add these as-is.
        while i < n and intervals[i][1] < newInterval[0]:
            result.append(intervals[i])
            i += 1

        # ---------- PHASE 2: Merge all OVERLAPPING intervals ----------
        # Once we exit Phase 1, intervals[i] (if any) might overlap newInterval.
        # They overlap as long as the current interval's START is <= newInterval's END.
        # We expand newInterval to cover all of them.
        while i < n and intervals[i][0] <= newInterval[1]:
            # Expand the start to the smaller of the two starts
            newInterval[0] = min(newInterval[0], intervals[i][0])
            # Expand the end to the larger of the two ends
            newInterval[1] = max(newInterval[1], intervals[i][1])
            i += 1
        # After merging, add the (possibly expanded) newInterval once.
        result.append(newInterval)

        # ---------- PHASE 3: Intervals that come AFTER newInterval ----------
        # All remaining intervals start strictly after newInterval ends.
        # Since the input is sorted, just append them in order.
        while i < n:
            result.append(intervals[i])
            i += 1

        return result
```

---

4. Complexity Analysis

Time Complexity: O(n) - We make a single pass through all `n` intervals. Each interval is visited exactly once across the three phases (the pointer `i` only moves forward). All operations inside (min, max, append) are O(1).

Space Complexity: O(n) *(for the output)* - The result array can hold up to `n + 1` intervals. If we exclude the output array (which is required by the problem), the auxiliary space is **O(1)** — we only use a few variables.

---

5. Alternative Approaches

A) Reference Solution (Single Loop with Early Return)

The provided reference combines the three phases into one loop using `if/elif/else`. It's elegant and uses an early `return` with slicing (`intervals[counter:]`) when it detects the first interval after `newInterval`. Same O(n) time. Slightly less explicit than the three-phase version, but functionally equivalent.

B) Binary Search + Merge — O(log n + k)

Since intervals are sorted, you could **binary search** to find the insertion boundaries (where `newInterval` starts overlapping and where it stops). Then merge only the overlapping region (`k` intervals) and splice the arrays. - Finding boundaries: O(log n) - Merging overlapping region: O(k) - However, **building the result still requires copying** the non-overlapping intervals, so overall it remains **O(n)** in practice. The binary search only helps locate the merge zone faster, not reduce total work.

C) Treat as General Merge Intervals

Insert `newInterval` into the list, re-sort (O(n log n)), then run the standard **Merge Intervals** algorithm. This is simpler conceptually but **slower (O(n log n))** because it discards the pre-sorted property. Not recommended here since the three-phase O(n) approach exploits the sorted input optimally.

Key Takeaways

- **Pattern**: Two-pointer / single-pass partitioning into "before / overlap / after" regions.
- **Overlap test**: `[a,b]` and `[c,d]` overlap ⟺ `a <= d and c <= b`.
- **Exploit sortedness**: It lets us handle each region with a simple linear scan and avoid re-sorting.

Codex Solution

1. Intuition

The intervals are already:

1. **Sorted by start**
2. **Non-overlapping**

So we do **not** need to sort again.

When inserting `newInterval`, every existing interval falls into one of three categories:

Case 1: Completely before `newInterval`

```
interval.end < newInterval.start
```

Example:

```
[1, 2]   newInterval = [5, 7]
```

This interval cannot overlap, so we add it directly to the result.

Case 2: Completely after `newInterval`

```
interval.start > newInterval.end
```

Example:

```
newInterval = [2, 4]   [6, 8]
```

Since intervals are sorted, once we find an interval after `newInterval`, all remaining intervals are also after it.

So we can:

1. Add `newInterval`
2. Add the rest of the intervals
3. Return immediately

Case 3: Overlaps with `newInterval`

```
interval.start <= newInterval.end
and
interval.end >= newInterval.start
```

Example:

```
interval = [3, 6]
newInterval = [4, 8]
```

They overlap, so we merge them:

```
newInterval.start = min(interval.start, newInterval.start)
newInterval.end   = max(interval.end, newInterval.end)
```

We keep expanding `newInterval` until no more overlaps exist.

---

2. Approach

Pseudocode

```
create empty result list

for each interval in intervals:

    if interval ends before newInterval starts:
        add interval to result

    else if interval starts after newInterval ends:
        add newInterval to result
        add all remaining intervals to result
        return result

    else:
        merge interval with newInterval
        update newInterval start and end

after loop:
    add newInterval to result

return result
```

---

3. Python Solution

```python
from typing import List

class Solution:
    def insert(
        self,
        intervals: List[List[int]],
        newInterval: List[int]
    ) -> List[List[int]]:
        # Result array that will contain the final non-overlapping intervals
        result = []

        # Iterate through all existing intervals
        for i, interval in enumerate(intervals):
            current_start = interval[0]
            current_end = interval[1]

            new_start = newInterval[0]
            new_end = newInterval[1]

            # Case 1:
            # Current interval is completely before newInterval.
            #
            # Example:
            # current = [1, 2], newInterval = [5, 7]
            #
            # Since current_end < new_start, there is no overlap.
            if current_end < new_start:
                result.append(interval)

            # Case 2:
            # Current interval is completely after newInterval.
            #
            # Example:
            # newInterval = [2, 4], current = [6, 8]
            #
            # Since intervals are sorted, all remaining intervals are also
            # after newInterval. So we can safely insert newInterval now,
            # append the rest, and return immediately.
            elif current_start > new_end:
                result.append(newInterval)

                # Add all remaining intervals starting from current index
                result.extend(intervals[i:])
```

```
                return result

            # Case 3:
            # Current interval overlaps with newInterval.
            #
            # Example:
            # current = [3, 6], newInterval = [4, 8]
            #
            # Merge them into a single expanded newInterval.
            else:
                newInterval[0] = min(new_start, current_start)
                newInterval[1] = max(new_end, current_end)

        # If we finished the loop, it means newInterval belongs at the end
        # or has merged with intervals up to the end.
        result.append(newInterval)

        return result
```

---

4. Complexity Analysis

Let `n` be the number of intervals.

Time Complexity: `O(n)`

We scan through the intervals at most once.

Even when we use:

```
result.extend(intervals[i:])
```

the total number of copied intervals is still at most `n`.

Therefore, the overall time complexity is:

```
O(n)
```

Space Complexity: `O(n)`

We create a new output list `result`.

In the worst case, it contains all original intervals plus the inserted interval.

Therefore, the space complexity is:

```
O(n)
```

If we ignore the output array, the extra auxiliary space is:

```
O(1)
```

---

5. Alternative Approaches

Alternative 1: Append and Merge All Intervals

A simpler but less optimal-looking method:

1. Append `newInterval` to `intervals`
2. Sort all intervals by start time
3. Merge intervals like in the classic "Merge Intervals" problem

```
intervals.append(newInterval)
sort intervals
merge overlapping intervals
```

Complexity

```
Time: O(n log n)
Space: O(n)
```

This works, but it ignores the useful fact that the original intervals are already sorted.

Alternative 2: Binary Search Insert Position

Because intervals are sorted, we could binary search for where `newInterval` starts.

Then insert it and merge around that position.

However, after insertion, merging can still take `O(n)` in the worst case, because `newInterval` may overlap many intervals.

Complexity

```
Time: O(n)
Space: O(n)
```

This is more complicated and usually not necessary for this problem.